

JavaScript 자료구조 기초

학습 체크리스트 (1/2)

- ☐ 객체(Object)의 정의(프로퍼티와 메서드) 및 키를 활용한 기본적인 CRUD 조작 수행 여부
- ☐ Object 클래스의 정적 메서드(`Object.keys()`, `values()`, `entries()`)를 활용한 객체 루프 순회 여부
- ☐ 배열(Array)의 순서적 특징 및 인덱스 기반 CRUD 메서드(push, pop, splice 등) 제어 여부

학습 체크리스트 (2/2)

☐ 배열 `sort()` 메서드의 문자열 사전식 정렬 문제점 인지 및 숫자 오름차순/내림차순 정렬 함수 작성 여부

☐ Map 객체의 특성(키 타입 다양성, size 속성) 및 기본 CRUD 메서드(`set()`, `get()`, `has()`, `delete()`) 구동 여부

☐ Set 객체의 중복 불허 속성 이해 및 배열 중복 원소 제거(`new Set()`) 테크닉 구현 여부

객체 (Object)

객체 (Object)

키(Key)와 값(Value)의 쌍으로 구성된 프로퍼티(Property)들의 집합이자, 현실 세계의 사물이나 개념을 속성과 행위(Method)로 추상화한 복합 자료형

- **현실의 모사:** 상태(속성)와 동작(행위)을 한 곳으로 결합하여 구조화함
- **식별 이름:** 값에 고유한 키(Key) 이름을 부여하여 직관적인 조회가 가능함
- **유연한 구성:** 실행 중에도 프로퍼티를 자유롭게 붙이고 뗄 수 있는 유연성을 제공함

객체의 구성 요소

- 프로퍼티 (Property)
 - 객체의 내부 상태나 고유한 값을 나타내는 키-값(Key-Value) 쌍
 - 키는 주로 문자열이 사용되며, 값은 자바스크립트의 모든 자료형 대입 가능
- 메서드 (Method)
 - 객체의 기능이나 행위를 나타내는 프로퍼티 안의 함수
 - 메서드 내부에서는 `this` 키워드를 사용하여 자신이 소속된 객체의 속성에 접근 가능

객체 구성 요소 실전 예시

```
1  const car = {
2    brand: "Hyundai", // 프로퍼티 (상태)
3    speed: 120,      // 프로퍼티 (상태)
4
5    drive() {        // 메서드 (동작)
6      // this를 통해 소속 객체의 brand와 speed를 획득함
7      return `${this.brand}가 시속 ${this.speed}km로 달림`;
8    }
9  };
10
11 console.log(car.drive()); // "Hyundai가 시속 120km로 달림"
```

객체의 기본적인 CRUD 조작

- 생성 (Create): 중괄호 리터럴(`{}`)에 직접 키와 값을 입력하거나 빈 객체 생성
- 조회 (Read): 점 표기법(`obj.key`) 또는 동적 접근용 대괄호 표기법(`obj['key']`) 적용
- 수정 (Update): 기존에 선언된 키에 접근하여 다른 값을 대입하여 갱신함
- 삭제 (Delete): `delete obj.key` 연산자를 명시하여 특정 키-값 세트를 영구 탈퇴 처리

객체 CRUD 조작 실전 예시

```
1  const user = { name: "민지" };
2
3  // 1. Create (새로운 속성 동적 생성)
4  user.age = 20;
5
6  // 2. Read (조회 - 점 및 대괄호 표기)
7  console.log(user.name); // "민지"
8  console.log(user["age"]); // 20
9
10 // 3. Update (수정)
11 user.age = 21;
12
13 // 4. Delete (삭제)
14 delete user.age;
15 console.log(user.age); // undefined (삭제 성공)
```


Object 정적 메서드를 활용한 데이터 추출

- `Object.keys(obj)`
 - 객체가 가지고 있는 모든 ****키(Key)****들을 모아 문자열 배열로 정돈하여 반환
- `Object.values(obj)`
 - 객체가 가지고 있는 모든 ****값(Value)****들만 쏙 빼내어 하나의 배열로 정돈하여 반환
- `Object.entries(obj)`
 - 객체의 **[키, 값]** 쌍을 통째로 이중 배열(`[ [Key, Value], ... ]`) 형태로 모아 반환
 - 객체 데이터에 대한 루프 순회 및 필터링 시 강력한 편의성을 제공함

Object 정적 메서드 실전 예시

```
1  const stats = { hp: 120, mp: 50 };
2
3  // 1. 키 배열 획득
4  console.log(Object.keys(stats));
5  // ["hp", "mp"]
6
7  // 2. 값 배열 획득
8  console.log(Object.values(stats));
9  // [120, 50]
10
11 // 3. [키, 값] 쌍 2차원 배열 획득
12 console.log(Object.entries(stats));
13 // [["hp", 120], ["mp", 50]]
```

배열 (Array)

배열 (Array)

순서가 있는 데이터의 컬렉션을 관리하는 선형 자료형으로, 대괄호(`[]`)를 이용해 선언하고 0부터 시작하는 정수 인덱스로 개별 값에 접근하는 특수 객체

- **순서적 인덱스:** 모든 원소는 순차적인 번호표(0, 1, 2...)를 부여받아 순서가 엄격하게 관리됨
- **동적 배열 크기:** 고정 크기 제한이 없으며 원소 추가 시 메모리가 동적으로 자동 늘어남
- **다양한 API:** 다양한 원소 삽입, 삭제, 가공 전용 기본 내장 메서드들을 완비함

배열의 기본적인 CRUD 조작

- 조회 (Read): `arr[인덱스]` 형태로 개별 요소 접근
- 맨 뒤 추가 / 제거: `push()` (추가) 및 `pop()` (제거) → 속도가 빠름
- 맨 앞 추가 / 제거: `unshift()` (추가) 및 `shift()` (제거) → 원소 재정렬로 다소 무거움
- 중간 정밀 제어 (Update & Delete):
 - `splice(시작인덱스, 삭제개수, 추가할원소...)` 를 통해 원하는 중간 지점 원소들을 완벽하게 교체 및 삽입 가능

배열 CRUD 조작 실전 예시

```
1  const fruits = ["사과", "바나나"];
2
3  // 1. Create (맨 뒤 원소 추가)
4  fruits.push("체리"); // ["사과", "바나나", "체리"]
5
6  // 2. Update (인덱스를 통한 특정 원소 값 교체)
7  fruits[1] = "메론"; // ["사과", "메론", "체리"]
8
9  // 3. Delete & Update (pop으로 뒤쪽 탈락 & splice로 정밀 교환)
10 fruits.pop(); // "체리" 제거 -> ["사과", "메론"]
11 fruits.splice(1, 1, "포도"); // 1번 메론 1개 지우고 "포도" 삽입
12 console.log(fruits); // ["사과", "포도"]
```

배열 정렬 (Sort)과 숫자 정렬의 위험성

- `sort()` 기본 아스키 정렬 문제
 - 아무 인자 없이 `sort()` 를 가동하면 원소들을 일시적으로 문자열로 취급하여 정렬함
 - 아스키 사전식 기준 때문에 숫자 `10` 이 `2` 보다 우선 배치되는 치명적인 오정렬 발생
- 비교 함수 주입의 의무화
 - 숫자 정렬 시에는 반드시 `**비교 콜백 함수 (a, b) => 반환값 **`을 공급해야 함
 - 오름차순: `(a, b) => a - b` (a가 크면 양수가 반환되어 b가 앞으로 이동)
 - 내림차순: `(a, b) => b - a`

배열 정렬 실전 예시

```
1  const numbers = [4, 2, 10, 25];
2
3  // 1. ⚠ 기본 sort() 오동작 (문자열 사전식 정렬 한계)
4  numbers.sort();
5  console.log(numbers); // [10, 2, 25, 4] (오작동!)
6
7  // 2. 올바른 숫자 오름차순 정렬 (a - b)
8  numbers.sort((a, b) => a - b);
9  console.log(numbers); // [2, 4, 10, 25]
10
11 // 3. 올바른 숫자 내림차순 정렬 (b - a)
12 numbers.sort((a, b) => b - a);
13 console.log(numbers); // [25, 10, 4, 2]
```

맵 (Map)

맵 (Map)

키와 값의 쌍을 저장하는 컬렉션으로, 일반 객체와 달리 문자열이나 심볼뿐만 아니라 객체나 함수를 포함한 모든 자료형을 키로 지정할 수 있는 ES6 신규 자료 구조

- **키의 유연성:** 일반 객체의 문자열 제약을 타파하여 순수 객체 자체도 키로 대입 가능
- **크기 즉시 획득:** `size` 속성으로 루프 없이 즉시 요소의 개수를 받아냄
- **순서 지향:** 데이터가 들어간 순서대로 저장과 순회가 깔끔하게 100% 보장됨

Map의 CRUD 메서드 실전 예시

```
1  const memberMap = new Map();
2
3  // 1. Create (set 메서드로 데이터 대입)
4  memberMap.set("id_01", "김철수");
5  memberMap.set("id_02", "이영희");
6
7  // 2. Read (get으로 획득, has로 유무 조사, size로 수량 파악)
8  console.log(memberMap.get("id_01")); // "김철수"
9  console.log(memberMap.has("id_03")); // false (존재하지 않음)
10 console.log(memberMap.size);          // 2
11
12 // 3. Delete (delete 메서드로 정밀 제거)
13 memberMap.delete("id_02");
14 console.log(memberMap.size);          // 1
```

셋 (Set)

셋 (Set)

중복된 값을 일절 불허하고 오직 유일무이한 값(Unique Value)들로만 구성되는 ES6 신규 순서 없는 컬렉션 자료 구조

- **유일성 장치**: 동일한 값을 여러 번 집어넣어도 조용히 무시하여 단 하나만 수렴함
- **고속 검색 판정**: 원소의 유무를 판정하는 `has()` 의 성능이 일반 배열 대비 극적으로 빠름 ($O(1)$)
- **기본 제어**: 값의 삽입은 `add()`, 존재 검사는 `has()`, 이탈 처리는 `delete()` 로 수행

Set의 CRUD 메서드 실전 예시

```
1  const colorSet = new Set();
2
3  // 1. Create (add 메서드로 데이터 수집)
4  colorSet.add("Red");
5  colorSet.add("Blue");
6  colorSet.add("Red"); // ⚠ 중복 값이므로 자동 거부됨
7
8  console.log(colorSet.size);      // 2 (중복된 Red는 수용되지 않음)
9  console.log(colorSet.has("Blue")); // true
10
11 // 2. Delete (delete 메서드로 정밀 이탈)
12 colorSet.delete("Red");
13 console.log(colorSet.has("Red")); // false
```

Set을 활용한 배열 중복 원소 제거

- 원스톱 정화 트릭

- 스프레드 연산자(`...`)와 `new Set(배열)` 조합의 시너지
- 배열 내 중복 요소를 한 줄의 코드로 완벽하고 안전하게 여과하여 순수 정화된 유니크 배열로 재가공 수집 가능

```
1  const duplicateNumbers = [1, 2, 2, 3, 3, 3, 4];
2
3  // Set에 대입하여 중복을 자동 여과하고, 스프레드로 다시 풀어 배열화!
4  const uniqueNumbers = [...new Set(duplicateNumbers)];
5
6  console.log(uniqueNumbers); // [1, 2, 3, 4]
```